**Piauí Instituto de Tecnologia – PIT**

**Disciplina**: Algoritmo, Estruturas de Dados e Programação

**Professor**: Joselito Junior

# Trabalho Final

## 1. Objetivo geral

- Desenvolver, em linguagem C, um sistema que simule as funcionalidades básicas de uma rede social, permitindo o cadastro e exclusão de usuários, a criação e remoção de amizades e a listagem de perfis e amizades.

- O projeto tem como principal finalidade consolidar o domínio de estruturas de dados do tipo struct, manipulação de vetores ou listas, ponteiros, modularização e boas práticas de programação em C.

- Será concedida pontuação extra aos alunos que implementarem armazenamento persistente (banco de dados ou arquivo) e/ou interface gráfica.

## 2. Requisitos funcionais (obrigatórios)

a) **Adicionar usuário**
   - Cada usuário deve ter pelo menos um identificador (id), nome completo, idade e um vetor (ou lista) de amigos.
   - O sistema deve impedir o cadastro de IDs duplicados.

b) **Remover usuário**
   - Ao remover um usuário, todas as suas amizades devem ser removidas também.

c) **Criar amizade**
   - A amizade é **bidirecional**: se Ana é amiga de Bruno, Bruno também é amigo de Ana.
   - Não deve ser possível adicionar uma amizade duplicada nem com o próprio usuário.

d) **Remover amizade**
   - Remover o vínculo de amizade entre dois usuários cadastrados.

e) **Listar amizades de um usuário**
   - Exibir o nome e o ID dos amigos de um determinado usuário.

f) **Listar todos os perfis cadastrados**
   - Exibir o ID, o nome, a idade e o número de amigos de cada usuário.

g) **Sair**
   - Antes de encerrar, liberar toda a memória alocada dinamicamente.

## 3. Estrutura de dados (requisito mínimo)

O projeto **deve obrigatoriamente utilizar structs em C** para representar os elementos do sistema.

- É permitido o uso de **vetores fixos** ou **alocação dinâmica** para os usuários.
- Cada usuário pode ter uma **lista (vetor) de ponteiros para amigos**, conforme o exemplo.
- O projeto deve demonstrar domínio do uso de **structs, ponteiros e passagem de parâmetros por referência**.

Os alunos que desejarem poderão implementar **listas encadeadas** ou **estruturas mais sofisticadas** (como listas de adjacência, hash tables ou árvores binárias) para aprimorar o desempenho e organização — recebendo **pontuação adicional** por isso.

## 4. Requisitos não funcionais

- **Linguagem:** C (padrão C99 ou superior).
- **Compilação:** gcc -Wall -Wextra -O2 programa.c -o mini_facebook
- **Modularização:** separar o código em arquivos .c e .h (por exemplo, usuario.c, usuario.h, rede.c, rede.h, main.c).
- **Boas práticas:** indentação consistente, comentários explicativos, mensagens claras.
- **Validação:** tratar entradas inválidas (usuário inexistente, amizade duplicada etc.).
- **Memória:** liberar todas as estruturas alocadas dinamicamente antes do encerramento.

## 5. Pontuação

| Critério | Pontos |
|---|---|
| Cadastro, exclusão e listagem de usuários | 2,5 |
| Criação e remoção de amizades | 2,5 |
| Listagens de amizades e perfis | 2,0 |
| Estruturação com structs, modularização e boas práticas | 2,0 |
| Liberação de memória e tratamento de erros | 1,0 |
| **Total** | **10,0** |

**Bônus (até +2,0)**

- **Banco de dados / arquivos** para armazenar e carregar usuários e amizades (+1,0)
- **Interface gráfica (GTK, Raylib, etc.)** ou interface textual estilizada (+1,0)

## 6. Entregáveis

1. Código-fonte completo (.c e .h) com README.md explicando como compilar e executar.
2. Relatório simples (até 3 páginas) explicando:
    - Estrutura de dados escolhida (struct e relacionamentos)
    - Principais funções e lógica de cada módulo
    - Dificuldades e soluções encontradas